

AI ASSISTED CODING ASS – 20.1

2303A51476

B-27

Task 1 - Input Validation Check

Task: Analyze an AI-generated Python login script for input validation vulnerabilities.

Instructions:

- Prompt AI to generate a simple username-password login program. • Review whether input sanitization and validation are implemented.
- Suggest secure improvements (e.g., using re for input validation).

Expected Output:

- A secure version of the login script with proper input validation.

Prompt:

Create a simple Python login program using predefined username and password.

Then:

- Identify input validation and security issues in the code.
- Improve it using proper validation (regex, length checks, and safe input handling).

Finally, provide:

- Original insecure code
- Brief explanation of vulnerabilities
- Secure improved version of the code

CODE:

```
# Original Insecure Code
def login():
    predefined_username = "admin"
    predefined_password = "password123"

    username = input("Enter username: ")
    password = input("Enter password: ")

    if username == predefined_username and password == predefined_password:
        print("Login successful!")
    else:
        print("Invalid credentials.")

if __name__ == "__main__":
    login()
```

Output:

```
Enter username: admin
Enter password: password123
Login successful!
```

```
Enter username: admin
Enter password: wrongpass
Invalid credentials.
```

```
# Updated Secure Version of the Code
import re
def secure_login():
    predefined_username = "admin"
    predefined_password = "StrongP@ssw0rd!"
    username = input("Enter username: ")
    password = input("Enter password: ")

    # Input validation
    if not re.match(r'^[a-zA-Z0-9_]+$', username):
        print("Invalid username. Only alphanumeric characters and underscores are allowed.")
        return

    if len(password) < 8:
        print("Password must be at least 8 characters long.")
        return

    if username == predefined_username and password == predefined_password:
        print("Login successful!")
    else:
        print("Invalid credentials.")

# To run the secure login function
if __name__ == "__main__":
    secure_login()
```

Output:

```
Enter username: admin
Enter password: StrongP@ssw0rd!
Login successful!
```

```
Enter username: admin
Enter password: Ab1@
Password must be between 8 and 30 characters.
```

Task 2 – SQL Injection Prevention

Task: Test an AI-generated script that performs SQL queries on a database.

Instructions:

- Ask AI to generate a Python script using SQLite/MySQL to fetch user details.
- Identify if the code is vulnerable to SQL injection (e.g., using string concatenation in queries).
- Refactor using parameterized queries (prepared statements).

Expected Output:

- A secure database query script resistant to SQL injection.

Prompt:

Create a Python script using SQLite (or MySQL) to fetch user details from a database based on user input.

Then:

- Analyze the code for SQL injection vulnerabilities (e.g., unsafe string concatenation).
- Refactor the code using parameterized queries (prepared statements) to prevent SQL injection. Finally, provide:
- Original vulnerable code
- Brief explanation of the vulnerability
- Secure version of the script

CODE:

```
# Original vulnerable code
import sqlite3
def get_user_details(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    # Vulnerable to SQL injection
    query = f"SELECT * FROM users WHERE username = '{username}'"
    cursor.execute(query)

    user_details = cursor.fetchone()
    conn.close()

    return user_details
```

Output:

```
('admin', 'admin@example.com', 'password123')
```

```
# Secure version of the script using parameterized queries
import sqlite3
def get_user_details_secure(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    # Using parameterized queries to prevent SQL injection
    query = "SELECT * FROM users WHERE username = ?"
    cursor.execute(query, (username,))

    user_details = cursor.fetchone()
    conn.close()

    return user_details
```

Output:

```
('admin', 'admin@example.com', 'password123')
```

None

Task 3 – Cross-Site Scripting (XSS) Check

Task: Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities.

Instructions:

- Ask AI to generate a feedback form with JavaScript-based output.
- Test whether untrusted inputs are directly rendered without escaping.
- Implement secure measures (e.g., escaping HTML entities, using CSP).

Expected Output:

- A secure form that prevents XSS attacks.

Prompt:

Create a simple HTML feedback form with JavaScript that displays user input on the page.

Then:

- Analyze the code for XSS vulnerabilities (e.g., unsafe use of `innerHTML`).
- Improve it by implementing secure practices such as escaping HTML, using safe DOM methods, and applying Content Security Policy (CSP).

Finally, provide:

- Original vulnerable code
- Brief explanation of the vulnerability
- Secure version of the form

CODE:

```
# Original vulnerable code
<!DOCTYPE html>
<html>
<head>
|   <title>Feedback Form</title>
</head>
<body>

<h2>Feedback Form</h2>

<form id="feedbackForm">
|   <textarea id="feedback" placeholder="Enter feedback"></textarea><br><br>
|   <button type="submit">Submit</button>
</form>

<div id="displayFeedback"></div>

<script>
document.getElementById("feedbackForm").addEventListener("submit", function(event) {
|   event.preventDefault();
|   let feedback = document.getElementById("feedback").value;
|   document.getElementById("displayFeedback").innerHTML = feedback;
});
</script>

</body>
</html>
```

Output:

Feedback Form

good

```
# Secure version of the form
<!DOCTYPE html>
<html>
<head>
  <title>Secure Feedback Form</title>
  <meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self';">
</head>
<body>

  <h2>Feedback Form</h2>

  <form id="feedbackForm">
    <textarea id="feedback" placeholder="Enter feedback"></textarea><br><br>
    <button type="submit">Submit</button>
  </form>

  <div id="displayFeedback"></div>

  <script>
function escapeHTML(str) {
  return str.replace(/[\&<>"]/g, function(char) {
    return {
      '&': '&amp;',
      '<': '&lt;',
      '>': '&gt;',
      '"': '&quot;',
      "'": '&#39;';
    }[char];
  });
}
  </script>
</body>
</html>
```

```
document.getElementById("feedbackForm").addEventListener("submit", function(event) {
  event.preventDefault();

  let feedback = document.getElementById("feedback").value.trim();

  if (feedback.length === 0) {
    alert("Feedback cannot be empty");
    return;
  }

  let safeFeedback = escapeHTML(feedback);

  let output = document.getElementById("displayFeedback");
  let p = document.createElement("p");

  p.textContent = safeFeedback;
  output.appendChild(p);
});
</script>
</body>
</html>
```

Output:

Secure version of the form

Feedback Form

nice

Submit

nice

Task 4 – Real-Time Application: Security Audit of AI-Generated Code

Scenario: Students pick an AI-generated project snippet (e.g., login form, API integration, or file upload).

Instructions:

- Perform a security audit to detect possible vulnerabilities.
- Prompt AI to suggest secure coding practices to fix issues.
- Compare insecure vs secure versions side by side.

Expected Output:

- A security-audited code snippet with documented vulnerabilities and fixes.

Prompt:

Generate a simple real-world code snippet (e.g., login system, API integration, or file upload).

Then:

- Perform a security audit to identify potential vulnerabilities (such as input validation issues, insecure storage, or injection risks).
- Suggest and implement secure coding practices to fix these issues.

Finally, provide:

- Original insecure code
- Identified vulnerabilities (brief explanation)
- Secure improved version of the code
- A comparison of insecure vs secure versions

CODE:

```
# Original insecure code
def login(username, password):
    # Insecure: Storing passwords in plain text
    users = {
        'user1': 'password123',
        'user2': 'mypassword'
    }
    if username in users and users[username] == password:
        return "Login successful"
    else:
        return "Login failed"
```

Output:

Login successful

Login Failed

```
# Secure improved version of the code
import bcrypt
def secure_login(username, password):
    # Secure: Storing hashed passwords using bcrypt
    users = {
        'user1': bcrypt.hashpw('password123'.encode('utf-8'), bcrypt.gensalt()),
        'user2': bcrypt.hashpw('mypassword'.encode('utf-8'), bcrypt.gensalt())
    }

    if username in users and bcrypt.checkpw(password.encode('utf-8'), users[username]):
        return "Login successful"
    else:
        return "Login failed"
```

Output:

Login successful

Invalid username format

Password must be at least 8 characters

Task 5 - Insecure Data Serialization Check

Task: Evaluate AI-generated code that uses pickle for saving user data.

Instructions:

- Ask AI to generate a script using pickle.load().
- Explain why untrusted pickle data is dangerous.
- Replace with safer formats like JSON.

Expected Output:

- Secure serialization code using JSON instead of pickle.

Prompt:

Create a Python script that saves and loads user data using pickle.

Then:

- Explain why using pickle.load() with untrusted data is dangerous.
- Replace it with a secure approach using JSON for serialization.

Finally, provide:

- Original insecure code
- Brief explanation of the risk
- Secure version using JSON

CODE:

```
# Original insecure code using pickle
import pickle
def save_user_data(user_data, filename):
    with open(filename, 'wb') as file:
        pickle.dump(user_data, file)

def load_user_data(filename):
    with open(filename, 'rb') as file:
        return pickle.load(file)
```

Output:

{'username': 'admin', 'password': 'password123'}

Hacked! Malicious code executed

```
# Secure version using JSON
import json
def save_user_data_secure(user_data, filename):
    with open(filename, 'w') as file:
        json.dump(user_data, file)

def load_user_data_secure(filename):
    with open(filename, 'r') as file:
        return json.load(file)
```

Output:

{'username': 'admin', 'password': 'password123'}

{'data': '<script>alert(1)</script>'}